A self-correcting step size plus axis search for escaping saddles

ENMG 616, saddle escape project. Brainstorm only: nothing is tested yet, and I have not yet checked the literature for similar methods.

The problem

Gradient descent (GD) steps $z_{k+1} = z_k - \eta \nabla f(z_k)$. At a saddle $\nabla f = 0$, so it stops even though f still curves down in one direction. The gradient is only a first-order picture of f. What it leaves out, the second-order curvature terms, is exactly what hides the way down at a saddle. Computing the Hessian would expose it, but that is expensive and a Hessian estimate has its own errors.

Part 1: let the prediction mismatch tune the step sizes

Intuition. At each step the gradient predicts how much f will drop. After the step I see how much it actually dropped. The gap between the two measures what the first-order picture missed, so I use it as feedback instead of computing curvature. Each direction gets its own step size (a diagonal matrix D, so $z_{k+1} = z_k - D\nabla f(z_k)$). Every iteration I nudge a step size up or down by a tiny amount, keep the change if the predicted-vs-actual gap gets smaller and undo it if not, with one safety check that f really went down.

Why it should help. On a quadratic, the perfect step along a direction with curvature λ is $1/\lambda$ (it lands in one step). The learned steps should drift toward $1/\lambda$ in each direction without ever computing λ. The ordinary GD step size is kept as an upper bound so the steps never get wilder than plain GD. Gentle directions, including a weak escape direction, should get bigger steps and stop crawling, while steep directions get smaller ones.

Part 2: if still stuck, search along the axes

Intuition. Starting very close to a saddle, ∇f is tiny and Part 1 has almost nothing to learn from. So when $\|\nabla f\|$ falls below a trigger, I place one probe along each coordinate axis inside a small ball, run a few steps from each, keep the probe that lowers f the most, and drop the ones that did not work (a compass search). Then I continue with Part 1 from there. Nothing is random: each probe reports back and I choose.

Example. $f = xy$ at (0,0) is a saddle. Probe to (1,0): f is still 0, but the gradient $(y, x) = (0, 1)$ is no longer zero, so GD slides down along $-y$ and f becomes negative. So f must be judged after a few steps, not right at the jump. If f goes up in every direction, I rotate the axes and retry, at most about 5 times. If it still never decreases, I treat the point as a minimum and stop.

Limits I already see

- Exactly on a saddle, $\nabla f = 0$ and $D \cdot 0 = 0$, so Part 1 cannot move anything. Part 2 has to do all the work there.
- Close to a saddle the gradient is tiny, so the predicted and actual changes are both tiny and the feedback is noisy. Part 1 may help most a bit away from the saddle.
- A diagonal D only sees curvature along the axes (for $f = xy$ the curvature along both axes is zero), which is why Part 2 rotates the axes. How many rotations are enough is a guess I would test.
- Extra cost: the probes and short runs at every stall, and extra evaluations of f for the feedback. I would count them in the work column.

What I would test

On $f_\gamma(x,y) = \tfrac{1}{2}x^2 - (\gamma/2)y^2 + \tfrac{1}{4}y^4$, compare GD, P-GD and GD-NC against this method: Part 1 only, Part 2 only, and both. Use small γ and starts near the saddle, exactly on it, and near the minimum, and report where it fails as well as where it helps.

Question for you: does this look like a reasonable choice for the “propose one modification” part of the project?